Lecture 9

HashiCorp Nomad

Workload orchestration — deploy and manage applications across on-prem and cloud

Based on Nomad Quick Start (HashiCorp Developer)

How Nomad Fits the Course

Lecture	Tool	Role
6–7	Docker + Terraform	Run containers locally; provision infra
8	Terraform	Baseline infra: buckets, databases
Airflow (earlier)	Orchestration	Schedule **DAGs** (pipelines)
9	**Nomad**	Schedule **long-running services & batch jobs** on a cluster

Idea: Terraform creates **machines and networks**; Nomad decides **which node runs which container** and keeps them healthy.

What Is Nomad?

Nomad is a **flexible scheduler and workload orchestrator** for deploying applications at scale.

- **Efficient resource usage** — Bin packing places workloads on client nodes
- **Self-healing** — Detects failed tasks and reschedules them
- **Zero-downtime deployments** — Rolling, blue/green, canary updates
- **Many workload types** — Docker, Java JARs, QEMU VMs, `exec`, custom drivers
- **Cross-platform** — Single binary; macOS, Windows, Linux; on-prem, cloud, edge
- **Declarative workflow** — One **job specification (jobspec)** for type, resources, services, region/datacenter

Nomad vs Kubernetes (High Level)

- **Nomad** — Simpler operational model; scheduler + jobspec; often paired with Consul/Vault
- **Kubernetes** — Richer ecosystem; steeper learning curve

Both solve: **run containers (and other workloads) reliably across many machines**. Data pipelines often need **batch + service** workloads — Nomad supports both natively.

Cluster Terms: Agents

agent — A Nomad process in **server** or **client** mode.

server — Manages jobs and clients, monitors tasks, **schedules** which tasks run on which nodes. Servers replicate state for HA.

client — Runs assigned tasks, registers with servers, waits for work. Often called a **node**.

Typical cluster: 3–5 server agents + **many client agents**.

dev agent — Single node, **server + client**, state **not** persisted to disk — great for learning; clean slate each run.

Operation Terms: Task → Job

task — Smallest unit of work. Executed by a **task driver** (`docker` , `exec` , …).

group — Tasks that run on the **same** Nomad client.

job — Top-level unit: your application definition (one or more groups).

job specification (jobspec) — HCL file describing job type, tasks, resources, constraints, services.

allocation — Mapping of a **task group** in a job to a **client node**. Nomad creates allocations when you run a job.

Job Types (Quick Start Emphasis)

Type	Use case
service	Long-lived until stopped (web API, Redis, workers)
batch	Runs until exit (ETL step, one-off migration)
parameterized batch	`nomad job dispatch` with **meta** (inputs)
periodic batch	**Cron**-like schedule inside Nomad
system	Run on every client (agents, log collectors)

Pytechco tutorial: Redis + web = **service**; setup = **parameterized batch**; employee simulation = **periodic batch**.

Application Workflow (Outside → Nomad)

1. **Build artifact** — CI (GitHub Actions, etc.) builds image/binary; push to registry (Docker Hub, GHCR).
2. **Nomad does not build artifacts** — It **pulls** them when scheduling.

Then with Nomad:

1. **Write jobspec** — Image, ports, `count`, services, constraints.
2. `nomad job run` — Nomad schedules allocations.
3. **Update** — Change code or jobspec; resubmit; Nomad rolls out (per update strategy).

Install Nomad CLI

- Download from Nomad downloads or package manager (e.g. **Homebrew** on macOS).
- This course assumes **Nomad ≥ 1.5** (matches Quick Start).

```
nomad -v
# Nomad v1.5.x or later
```

Create a Cluster (Local Dev)

Prerequisites: Docker running (for Docker driver in examples).

Dev agent (single machine; bind so containers can reach each other — as in HashiCorp tutorial):

```
sudo nomad agent -dev \
  -bind 0.0.0.0 \
  -network-interface='{{ GetDefaultInterfaces | attr "name" }}'
```

macOS + Docker Desktop: add `-data-dir="$HOME/nomad-dev-data"` (or run `lecture9/nomad-dev-macos.sh`) — avoids `/private/tmp/NomadClient` mount `permission denied`.

Another terminal:

```
export NOMAD_ADDR=http://localhost:4646
```

Deploy a Job: Docker Task (Pattern)

```
task "redis-task" {
  driver = "docker"
  config {
    image = "redis:7.0.7-alpine"
  }
}
```

- `driver` — How Nomad runs the task.
- `config.image` — Docker Hub by default; GHCR/Docker Hub URLs for private or org images.

Connection: Lecture 6–7 “Webserver in Docker”

Lecture 6/7: Terraform `docker_container` + nginx + HTML.

Lecture 9 assignment: **`batch`** job — **`echo`** **“Hello, world from Nomad!”** via Docker **busybox** (macOS-friendly; **`exec`** is Linux-only).

Optional: **`nginx-web.nomad.hcl`** — nginx **`service`** like Lecture 6.

Connection: Lecture 8 Terraform

- **Terraform** — S3 buckets, RDS, security groups (static infra).
- **Nomad** — Runs **consumers**: ETL workers, API servers, batch jobs that **read/write** that infra.

Optional pattern: Terraform outputs (subnet IDs, IAM) → Nomad clients in that VPC run pipeline workloads.

Pytechco Example (HashiCorp Tutorial)

- **pytechco-redis** — Service + **Nomad service** discovery (`redis-svc`).
- **pytechco-web** — Service on port **5000**; `nomadService` for Redis address.
- **pytechco-setup** — Parameterized batch; `nomad job dispatch -meta budget="..."` .
- **pytechco-employee** — Periodic batch; **cron** every few seconds.

Clone: `learn-nomad-getting-started` , branch/tag **v1.1**, `jobs/*.nomad.hcl` .

Useful CLI (From Quick Start)

```
nomad job run   <file>.nomad.hcl
nomad job stop  -purge <job>
nomad job status <job>
nomad alloc logs <alloc-id> <task-name>
nomad job dispatch -meta key=value <parameterized-job>
nomad node status
nomad job allocs <job>
```

Clean Up

```
nomad job stop -purge <job-name>
# Local dev: Ctrl+C on nomad agent
# Cloud: terraform destroy in learn-nomad repo
```

Troubleshooting (local dev)

What you see	Why	What to do
`cpu` **exhausted** / can't place	Dev node has no free **reserved** CPU (other jobs, tight defaults)	`nomad job stop -purge …`, `nomad system gc`, restart agent; lower `resources` in the jobspec
`missing drivers`	`exec` = **Linux only**	On **macOS**, use **Docker** jobspecs (`hello-world.nomad.hcl`), not `hello-world-exec`
`permission denied` / `NomadClient` under `/private/tmp`	`-dev` puts allocs under `/private/tmp`; Docker Desktop can't mount it	Use `-data-dir="$HOME/nomad-dev-data"` (or `./nomad-dev-macos.sh`) — `sudo` **alone does not fix this**

Summary

- **Nomad** schedules **tasks** on **clients**; **servers** coordinate and persist intent.
- **Jobspec** = declarative unit; **service** vs **batch** vs **periodic** / **parameterized**.
- Fits the course: **Terraform** provisions; **Nomad** runs **Docker**, `exec` (Linux), and other drivers at scale.

Next: Assignment — dev cluster + `hello-world` batch job + `nomad alloc logs`.

Assignment Preview

Install Nomad → dev agent → `nomad job run assignment/hello-world.nomad.hcl` → read logs (**Hello, world from Nomad!**).

See **LECTURE9_ASSIGNMENT_README.md**.